

Introduction

This is the exercise sheet for Lecture 3. During the lecture you saw how to systematically translate an ER diagram into relational tables: entities become tables, 1:N relationships become a foreign key on the “many” side, M:N relationships become a junction table, 1:1 relationships become a foreign key with a **UNIQUE** constraint, and weak entities get a composite primary key that includes their owner’s key. You also saw PostgreSQL’s core data types, **CREATE TABLE** syntax, the main constraint types (**PRIMARY KEY**, **FOREIGN KEY REFERENCES**, **NOT NULL**, **UNIQUE**, **CHECK**, **DEFAULT**), **ALTER TABLE**, and the three cascade rules for foreign keys (**CASCADE**, **SET NULL**, **RESTRICT**). In this session you will:

- warm up by matching short attribute descriptions to the right PostgreSQL data type,
- practice the 1:1-relationship translation rule on a small, fresh example,
- translate the full university ER diagram from Lecture 2 into complete SQL DDL,
- extend an existing schema with **ALTER TABLE**,
- reason about **CASCADE** vs. **RESTRICT** vs. **SET NULL** for a set of foreign keys, and
- start translating your own semester project’s ER diagram into SQL DDL.

You can find a ready-to-run, Docker-based PostgreSQL setup with both the library schema from the lecture’s live-coding walkthrough and the university schema from Exercise 3 below (each with matching seed data, in its own PostgreSQL schema) pre-loaded in the **extras/** folder that accompanies this exercise sheet (see **How-to-run-PostgreSQL.md**) – handy for experimenting with constraints and cascade behaviour hands-on.

1 Exercise 1: Warm-up – Picking the Right Data Type

For each attribute description below, pick the PostgreSQL data type that best fits it, from: **NUMERIC(p,s)**, **BOOLEAN**, **TEXT**, **TIMESTAMPTZ**, **BIGINT**. Briefly justify your choice.

1. A product’s price, which must be stored exactly, in whole cents (no rounding errors), with up to two decimal places.
2. Whether a customer has opted in to marketing emails: a plain yes/no flag.
3. The full text of a customer review, which could be a single word or several paragraphs – there is no reasonable fixed maximum length.
4. The exact moment an order was placed, including the time zone it was placed in (the company has customers across several time zones).
5. A social-media post’s view counter, which for a very popular post could exceed 2 billion.

Solution.

1. ***NUMERIC**(p, s), e.g. **NUMERIC**(8,2). Money must never be stored as a floating-point type (**REAL**/**DOUBLE PRECISION**), because those cannot represent all decimal fractions exactly and can introduce rounding errors. **NUMERIC**(p, s) stores an exact decimal value with p total digits and s of them after the decimal point.*
2. ***BOOLEAN**: a plain true/false value, exactly what **BOOLEAN** is for.*
3. ***TEXT**: unbounded-length free text. Unlike **VARCHAR**(n), **TEXT** has no length limit to choose (and in PostgreSQL specifically, **TEXT** and **VARCHAR** have identical performance, so there is no downside to using it here).*
4. ***TIMESTAMPTZ** (**TIMESTAMP WITH TIME ZONE**): a point in time where the time zone matters. Plain **TIMESTAMP** (without time zone) would leave the time zone ambiguous, which is exactly the problem here since customers are in different zones.*
5. ***BIGINT**: a whole number that can exceed the range of a regular **INTEGER** (about ± 2.1 billion). **BIGINT** supports values up to roughly $\pm 9.2 \times 10^{18}$.*

2 Exercise 2: Translating a 1:1 Relationship

Consider the following small scenario at a company:

Each **Employee** (**employee_id**, **name**) may be assigned at most one **ParkingSpot** (**spot_id**, **location**), and each **ParkingSpot** is assigned to at most one **Employee**. Not every employee has a parking spot, and not every parking spot is currently assigned.

This is a **1:1** relationship between **Employee** and **ParkingSpot** (with partial participation on both sides).

1. Write the two **CREATE TABLE** statements that implement this, including the foreign key that realizes the 1:1 relationship, and the constraint that stops the *same* foreign key value from being used by more than one row (i.e. what actually enforces the “at most one” on the referencing side).
2. Which table did you put the foreign key on, and would it have worked just as well on the other table instead? Briefly justify your choice.

Solution.

```
1 CREATE TABLE ParkingSpot (  
    spot_id    SERIAL PRIMARY KEY,  
    location   VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE Employee (  
    employee_id SERIAL PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    spot_id     INTEGER UNIQUE REFERENCES  
                ParkingSpot(spot_id)  
);
```

The **UNIQUE** constraint on **Employee.spot_id** is what turns an ordinary (1:N-shaped) foreign key into a genuine 1:1 relationship: without it, several employees could reference the same **spot_id**, which would make it 1:N (many employees, one spot) instead. **spot_id** is nullable (no **NOT NULL**) because participation is partial on the **Employee** side – not every employee has an assigned spot.

2. The foreign key was put on **Employee**, referencing **ParkingSpot**. It would have worked exactly as well the other way around (a **UNIQUE**, nullable **employee_id** column on **ParkingSpot** instead) – for a 1:1 relationship with partial participation on both sides, either side is a valid place for the foreign key; the choice is arbitrary and usually driven by which table feels more natural to query “from”.

3 Exercise 3: The University Schema, from ER to SQL

Recall the university ER model from Lecture 2's exercise sheet:

- **Teacher** (strong entity): `teacher_id` (key), `name` (composite: `first_name` + `last_name`), `email`.
- **Program** (strong entity): `program_id` (key – a natural, not surrogate, key: short codes like DAT, SWT, DVML), `name`.
- **Semester** (weak entity, owned by **Program** via the identifying relationship **Has**, 1:N, **Semester** total / **Program** partial): partial key `semester_number`, plus `start_date`.
- **Course** (strong entity): `course_id` (key), `title`, `ects`.
- **Student** (strong entity): `student_id` (key), `name` (composite: `first_name` + `last_name`), `email`.
- **Grade** – originally a relationship *with attributes* between **Student** and **Course**, but crow's foot notation cannot attach attributes to a relationship line, so (per the "Same Fact, in Crow's Foot" pattern from Lecture 2) it was **promoted to its own entity**: partial key `exam_date` (a student can sit the same course's exam more than once), plus `grade_value`. Owned jointly by **Student** and **Course**.
- **Teaches** (M:N, **Teacher**–**Course**, total participation on both sides), **Takes** (M:N, **Student**–**Course**, partial on both sides), and **Offered_In** (M:N, **Course**–**Semester**, total on both sides): none of these carry attributes of their own.
- **Coordinates** (1:1, **Teacher**–**Semester**, **Semester** total / **Teacher** partial): no attributes of its own.

Write the complete set of **CREATE TABLE** statements that implement this schema in PostgreSQL, in an order that respects foreign-key dependencies. For every table, include:

- an appropriate primary key (including composite primary keys where the ER model calls for one),
- all foreign keys, with an explicit **ON DELETE** behaviour (**CASCADE**, **RESTRICT**, or **SET NULL** – pick what you think is right, you will revisit this reasoning in Exercise 5),
- **NOT NULL** where a value is required, and **UNIQUE** / **CHECK** where relevant.

*Hint: **Semester** is a weak entity, the same way **Copy** was in the library schema – its primary key needs a foreign key baked into it. **Grade**'s stated key, (`student_id`, `course_id`, `exam_date`), already has three parts – unlike **Borrows/Loan** in the library schema, no surrogate key is needed here; think about why a plain extra column is enough this time but was not enough there. Finally, **Coordinates** is 1:1: which table gets the foreign key, and given that **Semester**'s participation in it is total, could **ON DELETE SET NULL** on that foreign key ever actually work?*

Solution.

```
-- name is composite in the ER diagram -> two columns,
-- same as Member/Author
-- in the library schema.
CREATE TABLE Teacher (
    teacher_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name  VARCHAR(50) NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Program (
    program_id VARCHAR(10) PRIMARY KEY,
    name       VARCHAR(100) NOT NULL
);

CREATE TABLE Course (
    course_id SERIAL PRIMARY KEY,
    title     VARCHAR(200) NOT NULL,
    ects      INTEGER NOT NULL
);

-- Student.program_id realizes Enrolled_In (1:N,
-- Program -> Student).
CREATE TABLE Student (
    student_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name  VARCHAR(50) NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL,
    program_id VARCHAR(10) NOT NULL REFERENCES Program
                (program_id) ON DELETE RESTRICT
);

-- Semester is a weak entity identified by (program_id,
-- semester_number).
-- coordinator_teacher_id realizes the 1:1 Coordinates
-- relationship: UNIQUE
-- enforces "at most one semester per teacher"; NOT
-- NULL enforces
-- Semester's total participation in Coordinates.
CREATE TABLE Semester (
    program_id          VARCHAR(10) NOT NULL
        REFERENCES Program(program_id) ON DELETE CASCADE
    ,
```

```
semester_number      INTEGER NOT NULL,
start_date           DATE NOT NULL,
coordinator_teacher_id INTEGER NOT NULL UNIQUE
REFERENCES Teacher(teacher_id) ON DELETE
RESTRICT,
PRIMARY KEY (program_id, semester_number)
);

-- Grade: the ER diagram's relationship-with-attributes
-- (Student <-> Course),
-- promoted to its own entity. (student_id, course_id)
-- alone can repeat -- a
-- student may retake an exam -- so exam_date is an
-- extra partial key, the
-- same weak-entity trick as Copy's copy_number in the
-- library schema.
CREATE TABLE Grade (
    student_id  INTEGER NOT NULL REFERENCES Student(
        student_id) ON DELETE RESTRICT,
    course_id   INTEGER NOT NULL REFERENCES Course(
        course_id) ON DELETE RESTRICT,
    exam_date   DATE NOT NULL,
    grade_value INTEGER NOT NULL CHECK (grade_value IN
        (-3, 0, 2, 4, 7, 10, 12)),
    PRIMARY KEY (student_id, course_id, exam_date)
);

-- Teaches (M:N, Teacher <-> Course) -> junction table.
CREATE TABLE TeacherCourse (
    teacher_id  INTEGER NOT NULL REFERENCES Teacher(
        teacher_id) ON DELETE CASCADE,
    course_id   INTEGER NOT NULL REFERENCES Course(
        course_id) ON DELETE CASCADE,
    PRIMARY KEY (teacher_id, course_id)
);

-- Takes (M:N, Student <-> Course) -> junction table.
CREATE TABLE StudentCourse (
    student_id  INTEGER NOT NULL REFERENCES Student(
        student_id) ON DELETE CASCADE,
    course_id   INTEGER NOT NULL REFERENCES Course(
        course_id) ON DELETE CASCADE,
    PRIMARY KEY (student_id, course_id)
);
```

```
-- Offered_In (M:N, Course <-> Semester) -> junction
-- table; the FK to
-- Semester is composite, since Semester's own primary
-- key is composite.
CREATE TABLE CourseSemester (
    course_id      INTEGER NOT NULL REFERENCES Course
                   (course_id) ON DELETE CASCADE,
    program_id     VARCHAR(10) NOT NULL,
    semester_number INTEGER NOT NULL,
    PRIMARY KEY (course_id, program_id, semester_number
    ),
    FOREIGN KEY (program_id, semester_number)
        REFERENCES Semester(program_id, semester_number)
        ON DELETE CASCADE
);
```

- *Semester* **CASCADEs** from *Program* (a weak entity, exactly like *Copy* from *Book*), and *TeacherCourse*, *StudentCourse*, and *CourseSemester* all **CASCADE** from both sides: each of these rows only records a fact about its two parents (“this teacher teaches this course”, “this course runs in this semester”), and that fact stops meaning anything once either parent is gone.
- *Student.program_id* and *Grade*’s two foreign keys use **RESTRICT** instead: a *Student* row and a *Grade* row are valuable records in their own right (an enrollment, an exam result) that should never silently disappear because a *Program* or *Course* was deleted – the same reasoning as *Loan* in the library schema.
- *Semester.coordinator_teacher_id* is also **RESTRICT**, but for a sharper reason: **CASCADE** would delete the semester itself just because its coordinator left, and **SET NULL** is not actually available here – the column is **NOT NULL** (*Semester*’s participation in *Coordinates* is total), so nulling it would immediately violate that constraint. **RESTRICT** is the only option that forces the real-world fix (assign a new coordinator first) instead of an automatic one that cannot actually happen.
- *CourseSemester*’s foreign key to *Semester* is the composite (*program_id*, *semester_number*), because *Semester* – being a weak entity – has a composite primary key; a foreign key must reference a full primary (or unique) key on the other side. Same principle as *Loan*’s composite foreign key to *Copy* in the library schema.
- *Grade* does **not** need a surrogate key, unlike *Loan*: the scenario already supplies a natural attribute that tells repeated attempts apart (*exam_date*), so extending the composite key to (*student_id*, *course_id*, *exam_date*) is enough. *Loan* had no such natural attribute to reach for – nothing about a repeat loan of the same copy by the same member was guaranteed to differ – which is precisely why that table needed a manufactured **SERIAL** key instead.

- *grade_value* gets a *CHECK* constraint restricting it to the seven values of the Danish grading scale (-3, 00, 02, 4, 7, 10, 12) – a fixed value set the ER diagram alone does not capture, but that the database can still enforce.

The exact schema above (with FK-safe *CREATE TABLE* ordering, comments, and matching seed data) is provided as *extras/sql/03_university_schema.sql* and *extras/sql/04_university_seed_data.sql*, and has been verified against a live PostgreSQL 16 instance: all nine *CREATE TABLE* statements run cleanly in this order, and the *CHECK/UNIQUE* constraints reject the invalid rows they are meant to reject.

4 Exercise 4: Extending the Schema with ALTER TABLE

Working against the `Student` and `Semester` tables from Exercise 3:

1. The university wants to start recording a student's phone number, but not every student has to provide one. Write the `ALTER TABLE` statement that adds this column to `Student`.
2. The university also wants to start recording when a semester *ends*, not just when it starts – but a semester that is still ongoing will not have an end date yet. Write the `ALTER TABLE` statement that adds this column to `Semester`, and a second `ALTER TABLE` statement that adds a named `CHECK` constraint guaranteeing that, whenever it is set, the end date is strictly after the start date.

Solution.

```
1 ALTER TABLE Student
  ADD COLUMN phone_number VARCHAR(20);
```

*No **NOT NULL** here, since the phone number is explicitly optional – leaving the column nullable (the default) is exactly what “not every student has to provide one” calls for.*

```
2 ALTER TABLE Semester
  ADD COLUMN end_date DATE;

ALTER TABLE Semester
  ADD CONSTRAINT end_after_start CHECK (end_date
    IS NULL OR end_date > start_date);
```

*end_date is nullable, for the same reason as **phone_number** above (an ongoing semester genuinely has none yet); the **IS NULL** escape hatch in the **CHECK** constraint is what keeps that nullability compatible with the ordering rule – without it, every ongoing semester's **NULL end_date** would fail the comparison and be rejected. Naming the constraint (**end_after_start**) is good practice: it makes the constraint easier to identify later (e.g. in an error message, or when dropping/altering it), instead of relying on PostgreSQL's auto-generated name.*

5 Exercise 5: Reasoning About Cascade Rules

For each of the following foreign-key relationships from the university schema (Exercise 3), decide whether it should use `ON DELETE CASCADE`, `ON DELETE RESTRICT`, or `ON DELETE SET NULL`, and justify your choice in one or two sentences. (No SQL needed here – just the choice and the reasoning.)

1. `Semester.program_id` $\rightarrow$ `Program.program_id`
2. `TeacherCourse.course_id` $\rightarrow$ `Course.course_id`
3. `Student.program_id` $\rightarrow$ `Program.program_id`
4. `Grade.student_id` $\rightarrow$ `Student.student_id`
5. `Semester.coordinator_teacher_id` $\rightarrow$ `Teacher.teacher_id`

Solution.

*Note: this is one **proposed** solution, not the only correct one. These choices depend on business requirements the scenario does not fully pin down, so a different – equally defensible – call is often possible for the same foreign key. In particular, almost anywhere **CASCADE** is chosen below, **RESTRICT** could reasonably be argued instead, if silently deleting the associated information turns out not to be wanted; what matters is that the choice is made deliberately and justified, not which specific option is picked.*

1. **CASCADE.** A *Semester* row only exists to describe a semester of a particular program; once that program record is deleted, the semester record is meaningless and should be removed along with it. There is no useful state a *Semester* could be left in with a dangling/nulled *program_id*.
2. **CASCADE.** A *TeacherCourse* row records that a specific teacher teaches a specific course – once the course is gone, that teaching fact no longer applies to anything and should disappear with it.
3. **RESTRICT.** A *Student* row is a valuable record in its own right, not merely descriptive data about its program. If a program is deleted while students are still enrolled in it, silently deleting (**CASCADE**) or anonymizing (**SET NULL**) those student records would erase real people's data. **RESTRICT** forces that decision to be made explicitly instead of happening automatically.
4. **RESTRICT.** Exactly the same reasoning as *Loan* in the library schema: a *Grade* row is a historical record (who sat which exam, and with what result). It should not vanish just because the student record is later deleted – the exam result is still meaningful and should be preserved, or the deletion explicitly blocked until it is dealt with.
5. **RESTRICT** – and this one is worth pausing on. **CASCADE** would be actively wrong here: it would delete the semester itself just because its coordinating teacher left. **SET NULL** looks

*tempting (“just leave the semester temporarily uncoordinated”), but it cannot actually work – `coordinator_teacher_id` is **NOT NULL**, because *Semester*’s participation in *Coordinates* is total, so the delete would simply be rejected by the **NOT NULL** constraint before **SET NULL** ever got a chance to act. **RESTRICT** is the only choice that matches reality: a semester’s coordinator must be reassigned first, before the old one can be deleted.*

*Unlike the library schema, **SET NULL** is not simply absent here by coincidence – item 5 shows why it fails: a foreign key’s participation constraint decides whether **SET NULL** is even on the table. **SET NULL** only ever works on a nullable foreign key, which means only on the partial-participation side of a relationship; total participation, as in *Semester*’s side of *Coordinates*, rules it out categorically, no matter how convenient it would be.*

6 Exercise 6: Translate the Domain You Modeled

In Lecture 2, Exercise 7 you modeled a domain of your own choosing as an ER diagram – your semester project, or any other domain you picked. Now translate that *same* ER diagram into complete SQL DDL. Keep working on whichever domain you chose then; do not switch to a different one here.

Requirements:

- Write a `CREATE TABLE` statement for every entity, in an order that respects foreign-key dependencies (or use deferred constraints if you prefer to create them in a different order).
- Make **all** constraints explicit: primary keys (including composite ones for any weak entity), foreign keys with an explicit `ON DELETE` behaviour, `NOT NULL` where required, and `UNIQUE` / `CHECK` wherever your model calls for them.
- Run your script against a real PostgreSQL database (e.g. the Docker setup from `extras/`, or your own installation) and confirm that every table is created without errors.
- Insert a handful of test rows into each table to confirm the constraints behave the way you intended (in particular, try at least one insert you *expect* to be rejected, e.g. a foreign key pointing at a row that does not exist, to confirm the constraint is actually there).

Submit your `.sql` script, together with a short note (a few sentences) on any place where you had to make a translation decision that was not fully determined by the ER diagram alone (e.g. a cascade rule, or how you handled a multi-valued attribute).

Checklist before you submit:

- ☐ Every entity from your ER diagram has a `CREATE TABLE` statement
- ☐ Every weak entity's table has the correct composite primary key
- ☐ Every 1:N relationship is a foreign key on the “many” side
- ☐ Every M:N relationship is a junction table with two foreign keys (plus a surrogate key if the same pair can legitimately recur)
- ☐ Every foreign key has an explicit, justified `ON DELETE` behaviour
- ☐ The script runs against a real PostgreSQL database without errors
- ☐ You tried at least one insert that should fail, and confirmed that it does

Solution.

*This exercise is **open-ended** – everyone's chosen domain is different, so there is no single fixed solution to check against. Check your own script against the checklist above: in particular, make sure every `ON DELETE` choice is deliberate and justified (not just left at the default), and that the constraint you deliberately tried to violate was actually rejected by the database, not silently accepted. If you want, you can check with your TA about the design.*

AI Disclosure

Claude, a large language model developed by Anthropic, was used to assist with structuring and grammar checking of this material. All generated text was critically reviewed, edited, and verified by the author.